AI Assisted Coding

Assignment 19.3

Name: Vivek

Hallticket:2303A52009

Task 1 - Python to C++ Conversion
Task:
Translate a simple Python class into C++ using AI assistance.
Instructions:
. Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
. Ensure proper handling of:
o Constructors
o Data types
Access specifiers
Compile and run both versions to verify output
Expected Output:
An equivalent working C++ class translated from Python.

Prompt and code:

```python
lab19.py > ...
""" convert a py code to C++code  generate  code to convert a Python class representing a Student
with attributes and methods into equivalent C++ code
• Ensure proper handling of:
o Constructors
o Data types
o Access specifiers
print the output of py code and the equivalent C++ code"""
class Student:
    def __init__(self, name: str, age: int, grade: float):
        self.name = name
        self.age = age
        self.grade = grade

    def display_info(self):
        print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")
# Example usage
student = Student("Alice", 20, 85.5)
student.display_info()
```

PY Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\lab19.py"
Name: Alice, Age: 20, Grade: 85.5
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

C++ Code:

```cpp
include <iostream>
include <string>
class Student {
private:
    std::string name;
    int age;
    float grade;
public:
    // Constructor
    Student(std::string name, int age, float grade) {
        this->name = name;
        this->age = age;
        this->grade = grade;
    }
    // Method to display student information
    void display_info() {
        std::cout << "Name: " << name << ", Age: " << age << ", Grade: " << grade << std::endl;
    }
};
int main() {
    Student student("Alice", 20, 85.5);
    student.display_info();
    return 0;
}
```

Task 2 – Java to Python Function Conversion
Task:
Convert a Java method that checks whether a number is prime into Python.
Instructions:
• Ask AI to translate a Java isPrime() method into Python
• Test the function with multiple inputs
• Ensure the Python version follows Pythonic syntax and logic
Expected Output:
A correct Python function equivalent to the original Java prime-
Prompt and code:

```
""" Convert a Java method that checks whether a number is prime into
Python."""
# java code
# public class PrimeChecker {
#     public static boolean isPrime(int n) {
#         if (n <= 1) {
#             return false;
#         }
#         for (int i = 2; i <= Math.sqrt(n); i++) {
#             if (n % i == 0) {
#                 return false;
#             }
#         }
#         return true;
#     }

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
# Example usage
number = 29
if is_prime(number):
    print(f"{number} is a prime number.")
else:    print(f"{number} is not a prime number.")
```

Output:

```
12 errors
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding\lab19.py"
29 is a prime number.
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task_03:
Task 3 – Pseudocode to Python Implementation
Task:
Translate a given pseudocode algorithm into Python using AI.
Instructions:
• Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
• Ask AI to convert it into executable Python code
• Validate correctness using sample input lists
Expected Output:
A working Python program that correctly implements the pseudocode logic.

```
""" convert the pseudocode for bubble sort algorithm into Python code.
BEGIN
  INPUT list of numbers
  n = length of list

  FOR i from 0 to n-1
    FOR j from 0 to n-i-2
      IF list[j] > list[j+1]
        SWAP list[j] and list[j+1]
      ENDIF
    ENDFOR
  ENDFOR

  OUTPUT sorted list
END"""
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Example usage
numbers = [64, 34, 25, 12, 22, 11, 90]
sorted_numbers = bubble_sort(numbers)
print("Sorted list:", sorted_numbers)
```

Output:

```
29 is a prime number.
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Use
Sorted list: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task:4
Translate a SQL query into a Pandas DataFrame operation in Python.
Instructions:
- **Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).**
- **Ask AI to generate equivalent Pandas code.**
- **Test the generated code on a sample DataFrame.**

Expected Output:
- **Equivalent Pandas query that retrieves the correct subset of data.**

CODE and PROMPT:

```
""" Convert the following SQL query into equivalent Pandas code in Python.

SQL Query:
SELECT name, salary FROM employees WHERE salary > 50000;

Also:
1. Create a sample Pandas DataFrame named 'employees' with columns (name, salary).
2. Apply the Pandas operation.
3. Print the result.

Ensure the output matches the SQL query result."""
import pandas as pd
# Create a sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'salary': [60000, 45000, 70000, 30000, 55000]
}
employees = pd.DataFrame(data)
# Apply the Pandas operation equivalent to the SQL query
result = employees[employees['salary'] > 50000][['name', 'salary']]
# Print the result
print(result)
```

Output:

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding> python -u "c:\Users\Vivek\OneDrive\Desktop\3-2\AI-A
      name  salary
0    Alice   60000
2  Charlie   70000
4      Eve   55000
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

Task_05:
Task 5 - Real-Time Application: Algorithm Translation Across Languages
Scenario:
A company maintains algorithms written in different programming languages.
Examples:
. Converting a Python searching algorithm into C Translating a Java sorting program into JavaScript
Instructions:
. Use AI to translate a selected algorithm between two programming languages
. Execute and test both versions
. Document translation challenges such as:
o Syntax differences
o Library support
Memory management
Expected Output:
Equivalent, tested code implementations in two different programming languages.

Prompt and code:
"""Translate the following Python algorithm into C.
Python Code:
 # Binary Search
def binary_search(arr, target):
left, right = 0, len(arr) - 1
while left <= right:
mid = (left + right) // 2
if arr[mid] == target:
return mid
elif arr[mid] < target:
left = mid + 1
else:
right = mid - 1
return -1
Also explain:
- Key syntax differences
- Memory management differences""""

Code:

```c
#include <stdio.h>
int binary_search(int arr[], int size, int target) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Avoids overflow
        if (arr[mid] == target) {
            return mid; // Return the index of the target
        } else if (arr[mid] < target) {
            left = mid + 1; // Search in the right half
        } else {
            right = mid - 1; // Search in the left half
        }
    }
    return -1; // Target not found
}
int main() {
    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 5;
    int result = binary_search(arr, size, target);
    if (result != -1) {
        printf("Element found at index: %d\n", result);
    } else {
        printf("Element not found in the array.\n");
    }
    return 0;
}
```